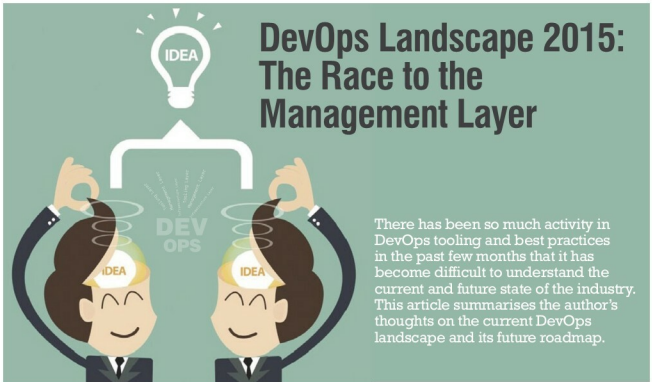




There has been so much activity in DevOps tooling and best practices in the past few months that it has become difficult to understand the current and future state of the industry. This article summarises the author's thoughts on the current DevOps landscape and its future roadmap.

The term 'DevOps' has many meanings, but for the purposes of this summary, it will be defined by its goal. Across organisations, the goal of DevOps is to turn application code into a running application through a repeatable, safe process.

There are two paradigms to provide a complete management layer for delivering applications.

Paradigm 1: Runtime configuration. In this view, the workflow from application code to running application is as follows: Provision a server; configure the server with dependencies and application code; and monitor the application and server's health with scheduled, continuous configuration management runs.

Paradigm 2: Immutable infrastructure. In this view, the workflow from application code to running application code is as follows: Package the application into a deployable artifact, such as a Docker container, AMI, jar, etc; then provision the server in the likeness of that artifact; and finally, monitor the application and server's health. It is very important to note that immutable infrastructure and configuration management are not mutually exclusive. Configuration management tools are often used to configure build-time artifacts.

The key difference between immutable infrastructure and configuration management paradigms is the point at which configuration occurs. In the immutable infrastructure view, configuration happens at build-time, before deployment. In the configuration management view, it happens at runtime, after deployment. To make a service change in the immutable infrastructure world, a new artifact is built, the old service

is destroyed, and a new service is brought up using the new artifact. The service is never changed once it has been deployed, and this is why this paradigm is called 'immutable' infrastructure. To make a service change in the configuration management world, the same server is updated with new service code. Again, these paradigms are not mutually exclusive, as configuration management tools are often used to help create build-time artifacts.

There's arguably a nascent third paradigm, which treats servers as a generic resource pool on top of which heterogeneous workloads are scheduled. This technically fits under the immutable infrastructure paradigm, but is unique enough to be defined independently. This paradigm is in active development as Mesos, Kubernetes, Docker Swarm and various other schedulers gain in popularity.

The management layer is the revenue opportunity

There are three basic layers in DevOps — infrastructure, tooling and management.

- The infrastructure layer comprises technology that provides virtualisation functionality to physical hardware to make compute, network and storage easily accessible. Examples of these technologies are Amazon's EC2 and Google Compute Engine in the public cloud arena, along with VMware and OpenStack in the private cloud arena.
- The tooling layer comprises technologies that make access and configuration of the infrastructure layer an easier process. These tools still require functional knowledge of the underlying infrastructure. Examples